```
const https = require("https");
const http  = require("http");

const GLITCH_KW = [
  "glitch","price error","price mistake","misprice","mispriced",
  "accidental","wrong price","pricing error","lowest ever","all time low",
  "record low","lowest price ever","price drop","flash sale","lightning deal",
  "coupon stack","deal alert","price match","checkout glitch",
  "insane deal","crazy deal","massive discount","huge discount",
];

const DEAD_KW = [
  "expired","out of stock","sold out","no longer available","deal ended",
  "price changed","link dead","removed","deleted","[expired]",
  "(expired)","deal is dead","ymmv expired","unavailable",
];

function fetchUrl(url, timeoutMs = 9000) {
  return new Promise((resolve, reject) => {
    const lib = url.startsWith("https") ? https : http;
    const req = lib.get(url, {
      headers: {
        "User-Agent": "Mozilla/5.0 (compatible; GlitchHunterBot/3.0)",
        "Accept": "application/rss+xml, application/xml, text/xml, application/js
        "Accept-Language": "en-US,en;q=0.9",
      },
    }, (res) => {
      if ([301,302,303,307,308].includes(res.statusCode) && res.headers.location)
        const next = res.headers.location.startsWith("http")
          ? res.headers.location
          : new URL(res.headers.location, url).href;
        fetchUrl(next, timeoutMs).then(resolve).catch(reject);
        return;
      }
      if (res.statusCode === 429) { reject(new Error("Rate limited")); return; }
      if (res.statusCode >= 400) { reject(new Error("HTTP " + res.statusCode)); r
      const chunks = [];
      res.on("data", c => chunks.push(c));
      res.on("end", () => resolve(Buffer.concat(chunks).toString("utf8")));
    });
    req.setTimeout(timeoutMs, () => { req.destroy(); reject(new Error("Timeout"))
    req.on("error", reject);
  });
}
```

```
function parseRSS(xml, source, color) {
  const items = [];
  const blocks = xml.match(/<item[\s\S]*?<\/item>/gi) || [];
  const now = Date.now();
  const FRESH = 72 * 60 * 60 * 1000;

  for (const block of blocks) {
    const get = (tag) => {
      const m =
        block.match(new RegExp("<" + tag + "[^>]*><!\\[CDATA\\[([\\s\\S]*?)\\]\\]
        block.match(new RegExp("<" + tag + "[^>]*>([\\s\\S]*?)<\\/" + tag + ">",
      return m ? m[1].replace(/<[^>]+>/g, " ").replace(/\s+/g, " ").trim() : "";
    };

    const title   = get("title");
    const desc    = get("description");
    const link    = get("link") || get("guid");
    const pubDate = get("pubDate") || get("dc:date") || get("pubdate");

    if (!title || title.length < 4) continue;

    if (pubDate) {
      const ts = new Date(pubDate).getTime();
      if (!isNaN(ts) && now - ts > FRESH) continue;
    }

    const combined = (title + " " + desc).toLowerCase();
    if (DEAD_KW.some(k => combined.includes(k))) continue;

    const isGlitch   = GLITCH_KW.some(k => combined.includes(k));
    const priceMatch = (title + " " + desc).match(/\$[\d,]+(?:\.\d{2})?/);
    const discMatch  = (title + " " + desc).match(/(\d{1,3})%\s*off/i);
    const ts         = pubDate ? (new Date(pubDate).getTime() || now) : now;

    items.push({
      id:       Buffer.from(title.slice(0, 40)).toString("base64").replace(/[^a-z
      title:    title.slice(0, 130),
      desc:     desc.slice(0, 300),
      link:     link || "",
      extLink:  null,
      source,   color,
      isGlitch,
      price:    priceMatch ? priceMatch[0] : null,
      discount: discMatch  ? parseInt(discMatch[1]) : null,
      timestamp: ts,
      score: null, comments: null, flair: null,
```

```
    });
  }
  return items;
}

const RSS_SOURCES = [
  { url: "https://slickdeals.net/newsearch.php?mode=frontpage&searcharea=deals&q=
  { url: "https://slickdeals.net/newsearch.php?mode=frontpage&searcharea=deals&q=
  { url: "https://slickdeals.net/newsearch.php?mode=frontpage&searcharea=deals&q=
  { url: "https://slickdeals.net/newsearch.php?mode=frontpage&searcharea=deals&q=
  { url: "https://slickdeals.net/newsearch.php?mode=frontpage&searcharea=deals&rs
  { url: "https://www.dealnews.com/c142/Computers/?srcval=rss_main",
  { url: "https://www.dealnews.com/c232/Electronics/?srcval=rss_main",
  { url: "https://www.dealnews.com/c196/Home-Garden/?srcval=rss_main",
  { url: "https://www.dealnews.com/c238/Clothing-Accessories/?srcval=rss_main",
  { url: "https://9to5toys.com/feed/",
  { url: "https://9to5mac.com/feed/",
  { url: "https://9to5google.com/feed/",
  { url: "https://techbargains.com/feed/",
  { url: "https://bensbargains.com/feed/",
  { url: "https://www.bradsdeals.com/blog/feed",
  { url: "https://www.thepennyhoarder.com/feed/",
  { url: "https://camelcamelcamel.com/top_drops/feed?n=25",
  { url: "https://www.hotukdeals.com/rss/deals",
  { url: "https://www.nytimes.com/wirecutter/deals/feed/",
];

const REDDIT_ID     = process.env.REDDIT_CLIENT_ID     || "";
const REDDIT_SECRET = process.env.REDDIT_CLIENT_SECRET || "";
const UA            = "GlitchHunterBot/3.0";
let rdCache         = { token: null, exp: 0 };

async function getRedditToken() {
  if (rdCache.token && Date.now() < rdCache.exp) return rdCache.token;
  if (!REDDIT_ID || !REDDIT_SECRET) return null;
  try {
    const auth = Buffer.from(REDDIT_ID + ":" + REDDIT_SECRET).toString("base64");
    const body = "grant_type=client_credentials";
    const raw  = await new Promise((resolve, reject) => {
      const req = https.request({
        hostname: "www.reddit.com",
        path: "/api/v1/access_token",
        method: "POST",
        headers: {
          "Authorization":  "Basic " + auth,
          "Content-Type":   "application/x-www-form-urlencoded",
          "Content-Length": Buffer.byteLength(body),
```

```
          "User-Agent":     UA,
        },
      }, (res) => {
        let d = "";
        res.on("data", c => d += c);
        res.on("end", () => resolve(d));
      });
      req.setTimeout(7000, () => { req.destroy(); reject(new Error("Timeout")); }
      req.on("error", reject);
      req.write(body);
      req.end();
    });
    const j = JSON.parse(raw);
    if (!j.access_token) return null;
    rdCache = { token: j.access_token, exp: Date.now() + (j.expires_in - 60) * 10
    return j.access_token;
  } catch (e) {
    return null;
  }
}

async function fetchReddit(sub, query, sort, limit) {
  sort  = sort  || "new";
  limit = limit || 25;
  try {
    const token = await getRedditToken();
    const base  = token ? "https://oauth.reddit.com" : "https://www.reddit.com";
    const hdrs  = token
      ? { "Authorization": "Bearer " + token, "User-Agent": UA }
      : { "User-Agent": UA };

    const path = query
      ? "/r/" + sub + "/search.json?q=" + encodeURIComponent(query) + "&sort=" +
      : "/r/" + sub + "/" + sort + ".json?limit=" + limit;

    const raw  = await fetchUrl(base + path);
    const json = JSON.parse(raw);
    const posts = (json && json.data && json.data.children) ? json.data.children
    const now   = Date.now();
    const FRESH = 72 * 60 * 60 * 1000;

    return posts.map(function(p) {
      const d = p.data;
      if (!d || !d.title) return null;
      const ts = d.created_utc ? d.created_utc * 1000 : now;
      if (now - ts > FRESH) return null;
      const combined = ((d.title || "") + " " + (d.selftext || "")).toLowerCase()
```

```
      if (DEAD_KW.some(k => combined.includes(k)) && (d.score || 0) < 10) return
      const isGlitch   = GLITCH_KW.some(k => combined.includes(k));
      const priceMatch = (d.title + " " + (d.selftext || "")).match(/\$[\d,]+(?:\
      const discMatch  = (d.title + " " + (d.selftext || "")).match(/(\d{1,3})%\s
      const extLink    = d.url && d.url.startsWith("http") && !d.url.includes("re
      return {
        id:       d.id || "",
        title:    (d.title || "").slice(0, 130),
        desc:     (d.selftext || "").replace(/\n+/g, " ").slice(0, 300),
        link:     "https://reddit.com" + (d.permalink || ""),
        extLink,
        source:   "r/" + sub,
        color:    "#ff6314",
        isGlitch,
        price:    priceMatch ? priceMatch[0] : null,
        discount: discMatch  ? parseInt(discMatch[1]) : null,
        timestamp: ts,
        score:    d.score        || 0,
        comments: d.num_comments || 0,
        flair:    d.link_flair_text || null,
      };
    }).filter(Boolean);
  } catch(e) {
    return [];
  }
}

const REDDIT_SOURCES = [
  { sub: "deals",          query: "",            sort: "hot" },
  { sub: "deals",          query: "amazon",      sort: "new" },
  { sub: "deals",          query: "glitch",      sort: "new" },
  { sub: "buildapcsales",  query: "",            sort: "hot" },
  { sub: "buildapcsales",  query: "price error", sort: "new" },
  { sub: "PCDeals",        query: "",            sort: "hot" },
  { sub: "PCDeals",        query: "glitch",      sort: "new" },
  { sub: "frugal",         query: "amazon",      sort: "new" },
  { sub: "DealAlert",      query: "",            sort: "new" },
  { sub: "amazondealsusa", query: "",            sort: "new" },
  { sub: "GoodValue",      query: "",            sort: "hot" },
  { sub: "SaleHunters",    query: "",            sort: "new" },
];

exports.handler = async function(event) {
  const headers = {
    "Access-Control-Allow-Origin":  "*",
    "Access-Control-Allow-Methods": "GET, OPTIONS",
    "Content-Type":                 "application/json",
```

```
  "Cache-Control":                "public, max-age=1800",
};

if (event.httpMethod === "OPTIONS") {
  return { statusCode: 200, headers, body: "" };
}

try {
  const [rssResults, rdResults] = await Promise.all([
    Promise.allSettled(
      RSS_SOURCES.map(s =>
        fetchUrl(s.url)
          .then(xml => parseRSS(xml, s.name, s.color))
          .catch(() => [])
      )
    ),
    Promise.allSettled(
      REDDIT_SOURCES.map(s => fetchReddit(s.sub, s.query, s.sort))
    ),
  ]);

  const rssItems = rssResults.flatMap(r => r.status === "fulfilled" ? r.value :
  const rdItems  = rdResults.flatMap(r  => r.status === "fulfilled" ? r.value :
  let all = [...rssItems, ...rdItems];

  const seen = new Set();
  all = all.filter(d => {
    if (!d || !d.title) return false;
    const key = d.title.toLowerCase().replace(/[^a-z0-9]/g, "").slice(0, 40);
    if (seen.has(key)) return false;
    seen.add(key);
    return true;
  });

  all.sort((a, b) => b.timestamp - a.timestamp);

  const glitches = all.filter(d => d.isGlitch);
  const deals    = all.filter(d => !d.isGlitch);
  const sourceCounts = {};
  all.forEach(d => { sourceCounts[d.source] = (sourceCounts[d.source] || 0) + 1

  return {
    statusCode: 200,
    headers,
    body: JSON.stringify({
      ok: true,
      ts: new Date().toISOString(),
```

```
        redditOn: !!(REDDIT_ID && REDDIT_SECRET),
        stats: {
          total: all.length, glitches: glitches.length,
          deals: deals.length, sources: Object.keys(sourceCounts).length,
          sourceCounts,
        },
        glitches, deals,
      }),
    };
  } catch (err) {
    return {
      statusCode: 500,
      headers,
      body: JSON.stringify({ ok: false, error: err.message }),
    };
  }
};
```